FE-5680A Integration Requirements and Phased Implementation Specification

Raspberry Pi + Teensy + ZED-F9T timing system
Working specification for hardware, firmware, software, telemetry, dashboard, and holdover integration

Truth source	Primary PPS	Measurement engine	Oscillator role
ZED-F9T	ZED PPS	Teensy	FE-5680A flywheel / holdover

Design guardrails

- ZED remains the sole GNSS and timing truth source.
- Raspberry Pi chrony remains disciplined by ZED PPS unless explicitly changed later.
- Teensy remains the timing measurement and oscillator-control engine.
- Piksi remains PPS comparison only.
- FE-5680A is integrated first as a measured oscillator, later as a disciplined flywheel and holdover source.

1. Document purpose

This document defines the project goals, operating concept, architecture rules, functional requirements, non-functional requirements, telemetry requirements, dashboard requirements, and phased implementation strategy for integrating an FE-5680A rubidium oscillator into the existing Raspberry Pi + Teensy + ZED-F9T timing system.

Use this specification to guide hardware integration, firmware development, Raspberry Pi software changes, database and dashboard updates, validation, and later holdover / discipline-loop development.

2. System context

2.1 Existing system

The current time server consists of a ZED-F9T GNSS receiver, Raspberry Pi running chrony and serving NTP, Teensy timing measurement and analytics engine, Piksi retained as PPS comparison reference, and a dashboard / SQLite telemetry pipeline.

2.2 Current design intent

Current timing architecture rules:

2.3 Proposed FE-5680A role

The FE-5680A shall be integrated as a local stable oscillator, a measurable frequency source, a later disciplined flywheel oscillator, and a later holdover source during loss of valid GNSS timing.

The FE-5680A shall not initially be used as GNSS truth, primary PPS truth, or a replacement for ZED in the Pi chrony discipline path.

3. Project objective

Integrate the FE-5680A into the existing timing system as a measured, observable, and later disciplined local oscillator that improves local stability and enables holdover, while preserving ZED-F9T as GNSS and PPS truth, Raspberry Pi chrony discipline from ZED PPS, Teensy as the timing measurement and control engine, and Piksi as comparison-only PPS reference.

The system should incorporate ZED timing uncertainty information, including quantization error (qErr) where beneficial, to improve phase / frequency estimation and avoid over-correcting the FE-5680A based on PPS quantization artifacts.

4. Scope

4.1 In scope

4.2 Out of scope for initial phases

5. Guiding architecture principles

5.1 Truth hierarchy

5.2 Separation of concerns

The system should clearly separate GNSS truth data, timing measurement data, oscillator telemetry, control-loop state, and holdover state.

5.3 Observability

All major FE states, measurements, and control values shall be visible in telemetry and dashboard outputs.

5.4 Safety

Any FE failure, invalidity, or offline condition shall not break the current ZED-based time server operation.

6. Concept of operations

6.1 Normal tracking mode

In normal tracking mode, ZED provides GNSS timing truth, ZED PPS marks the primary second boundary, Raspberry Pi remains disciplined by ZED PPS through chrony, Teensy measures FE behavior relative to ZED PPS, Teensy receives supporting ZED timing context from the Pi bridge, and FE may later be slowly steered toward long-term agreement with ZED.

6.2 Comparison mode

In comparison mode, Piksi PPS is measured as an independent comparison reference, is not used as steering truth for FE, and is not displayed as GNSS truth on the dashboard.

6.3 Holdover mode

In holdover mode, valid ZED timing is lost or invalid, FE remains running as the local oscillator, steering is frozen or transitions to holdover behavior, system estimates accumulated time uncertainty, and the dashboard indicates degraded timing confidence explicitly.

7. Top-level project goals

- ZED-F9T is the primary GNSS and timing truth source.
- ZED PPS is the primary PPS source.
- ZED PPS feeds both Raspberry Pi and Teensy.
- Raspberry Pi runs chrony and serves NTP.

- Teensy is the timing measurement and analytics engine.
- Piksi is retained only as a PPS comparison reference.
- FE-5680A is to be added as a local oscillator and later holdover / discipline element.

4.1 In scope

- FE-5680A power and signal integration
- FE output measurement by Teensy
- FE telemetry generation
- database storage for FE metrics
- dashboard visualization of FE state
- manual FE tuning support
- later closed-loop discipline
- later holdover state estimation
- qErr-aware timing estimation support

4.2 Out of scope for initial phases

- replacing ZED as truth source
- replacing Pi chrony ZED PPS discipline path
- using FE as direct NTP truth for the Raspberry Pi
- using Piksi as steering truth
- presenting FE as GNSS telemetry

5.1 Truth hierarchy

- ZED-F9T is the sole GNSS truth source.
- ZED PPS is the primary second marker.
- Teensy is the timing measurement and control engine.
- FE-5680A is the disciplined flywheel and holdover candidate.
- Piksi is comparison-only and not truth.

7. Top-level project goals

Goal	Name	Intent
G1	Preserve system truth hierarchy	FE integration must not alter or blur existing source-of-truth rules.
G2	Add a stable local oscillator	FE-5680A should provide a

Goal	Name	Intent
		measurable, stable local oscillator that improves local timing stability and supports later holdover.
G3	Enable disciplined oscillator operation	System should estimate FE frequency/phase error relative to ZED and later apply slow steering corrections.
G4	Enable holdover capability	FE-5680A should later support degraded but stable operation during temporary loss of valid GNSS/PPS truth.
G5	Maintain strong observability	System should expose FE status, drift, control, and holdover readiness clearly in telemetry and dashboard outputs.
G6	Use qErr-aware estimation where beneficial	System should use qErr and related timing uncertainty information to improve phase/frequency estimation and reduce control-loop overreaction to PPS quantization artifacts.

8. Functional requirements

8.1 Architecture requirements

ID	Requirement
FR-A1	The system shall preserve ZED-F9T as the sole GNSS timing truth source.
FR-A2	The system shall preserve ZED PPS as the primary PPS reference for both Raspberry Pi timing discipline and Teensy measurement alignment.
FR-A3	The system shall preserve Teensy as the primary timing measurement and oscillator-control engine.

ID	Requirement
FR-A4	The system shall preserve Piksi as a comparison-only PPS reference and shall not use Piksi as GNSS truth or FE steering truth.
FR-A5	The FE-5680A shall be treated as a disciplined local oscillator and holdover source, not as an independent truth source.

8.2 Hardware integration requirements

ID	Requirement
FR-HW1	The system shall provide stable power to the FE-5680A within its required operating range.
FR-HW2	The system shall provide appropriate grounding and signal integrity practices for FE integration.
FR-HW3	The FE output shall be conditioned as needed so that it can be safely and reliably measured by Teensy logic inputs.
FR-HW4	The FE control input path shall be protected against overvoltage, invalid startup conditions, and out-of-range commands.
FR-HW5	The system shall allow the FE output to be measured continuously during normal operation.
FR-HW6	The system should support later addition of control-voltage generation and monitoring.

8.3 Measurement requirements

ID	Requirement
FR-M1	The Teensy shall measure FE oscillator behavior relative to ZED PPS.
FR-M2	The system shall compute at least one FE frequency error estimate referenced to ZED timing.
FR-M3	The system shall determine FE validity state based on continuity, signal plausibility, and

ID	Requirement
	measurement sanity checks.
FR-M4	The system shall distinguish FE warmup from steady-state operation.
FR-M5	The system should estimate FE drift over time.
FR-M6	The system should support averaging and filtering to reduce noise in FE frequency estimation.
FR-M7	The system shall not require FE telemetry to overwrite or replace existing timing error fields used for ZED/Teensy timing summary.

8.4 qErr-aware estimation requirements

ID	Requirement
FR-Q1	The system shall ingest ZED timing-context data required to improve FE phase/frequency estimation.
FR-Q2	Where available, the system shall ingest ZED quantization error data or equivalent timing-uncertainty metadata through the Pi-to-Teensy path or an equivalent telemetry path.
FR-Q3	The control estimator shall account for qErr or equivalent timing uncertainty when interpreting short-term PPS residuals.
FR-Q4	The system shall avoid large steering corrections based on single residual samples when timing uncertainty is high.
FR-Q5	The system should reduce the weight of samples whose qErr or uncertainty exceeds configured thresholds.
FR-Q6	The system should log or expose estimator confidence status for diagnostics.

8.5 FE control and discipline requirements

ID	Requirement
FR-C1	The system shall support manual FE control output for characterization and tuning tests.
FR-C2	The system shall later support closed-loop FE steering using ZED-referenced timing error estimates.
FR-C3	The FE steering loop shall use slow update rates and bounded corrections to avoid chasing PPS noise or quantization artifacts.
FR-C4	The system shall support configurable gains, limits, and update rates for the steering loop.
FR-C5	The system shall prevent unsafe or implausible FE control commands.
FR-C6	The system shall expose FE discipline state in telemetry and dashboard outputs.
FR-C7	The system should preserve the last valid steering state when entering holdover.

8.6 Holdover requirements

ID	Requirement
FR-HO1	The system shall detect loss or invalidity of ZED timing reference.
FR-HO2	Upon loss of valid ZED timing, the FE control subsystem shall enter a defined holdover mode.
FR-HO3	During holdover, the system shall estimate accumulated timing uncertainty or expected time error.
FR-HO4	The dashboard shall clearly indicate holdover state and degraded timing confidence.
FR-HO5	The system should estimate holdover readiness before holdover entry.
FR-HO6	The system shall distinguish between tracking, locked, holdover, and fault states.

8.7 Telemetry, database, and API requirements

ID	Requirement
FR-T1	The system shall store FE-related telemetry in the database using FE-specific fields.
FR-T2	The database schema shall preserve existing ZED, Teensy, and Piksi data paths without ambiguity.
FR-T3	The system shall expose FE telemetry through the dashboard API in a dedicated manner.
FR-T4	The system shall not mix FE telemetry into GNSS-only fields.
FR-T5	The system shall preserve existing dashboard truth semantics for calibrated phase error, 10-minute RMS jitter, raw phase error, Piksi-ZED comparison, and GNSS health.
FR-T6	The system should expose qErr-aware status, confidence, or weighting diagnostics for debugging.

8.8 Dashboard and UI requirements

ID	Requirement
FR-D1	The dashboard shall display FE status in a dedicated oscillator or holdover section.
FR-D2	The dashboard shall not present FE as the primary timing truth source.
FR-D3	The dashboard shall not mix FE data into GNSS health panels.
FR-D4	The dashboard shall preserve current top timing summary emphasis on corrected timing rather than raw phase alone.
FR-D5	The dashboard shall display FE validity, state, warmup status, and key measured error values.
FR-D6	The dashboard should later display FE control values, discipline state, and holdover readiness.

ID	Requirement
FR-D7	The dashboard should remain deterministic and rule-based when generating narrative analysis of FE status.

8.9 Safety and robustness requirements

ID	Requirement
FR-S1	The system shall fail safe if FE measurement becomes invalid.
FR-S2	The system shall fail safe if FE control telemetry is missing or implausible.
FR-S3	The system shall preserve current ZED/Pi timing operation if the FE subsystem is offline or faulty.
FR-S4	The system shall log or expose invalid measurement conditions for troubleshooting.
FR-S5	The system shall clearly distinguish among offline, warmup, measure-only, manual-tune, acquire, locked, holdover, and fault states.

9. Non-functional requirements

- NFR-1 Stability: The FE discipline loop should favor stability over aggressiveness.
- NFR-2 Observability: All major FE states and control values should be visible in telemetry and dashboard outputs.
- NFR-3 Modularity: The FE subsystem should be separable from the current ZED-based baseline without breaking core timing operation.
- NFR-4 Maintainability: Implementation should follow the existing workflow of confirming live runtime paths, editing live copies as needed, syncing to repo copies, refreshing snapshot, and committing changes.
- NFR-5 Traceability: Each major FE feature should map to explicit telemetry fields, database fields, API values, dashboard elements, and validation tests.
- NFR-6 Defensive rendering: Invalid, missing, or sentinel FE values should be sanitized before visualization.

10. FE subsystem state model

State	Meaning
OFFLINE	FE not present or not measured
POWERING	Startup period before measurement validity
WARMUP	Running but not yet trusted for stability
MEASURE_ONLY	Measured continuously but not steered
MANUAL_TUNE	Manual EFC experiments or calibration
ACQUIRE	Attempting to converge under closed-loop control
LOCKED	Disciplined and stable
HOLDOVER	Truth unavailable, oscillator free-running with retained control state
FAULT	Unsafe or implausible condition detected

11. Initial telemetry and database field plan

11.1 Early-phase FE telemetry

- fe_valid
- fe_state
- fe_count
- fe_freq_error_ppb
- fe_freq_error_smoothed_ppb
- fe_warmup_seconds

11.2 Mid-phase fields

- fe_drift_ppb_per_hr
- fe_control_value
- fe_control_voltage
- fe_control_valid

11.3 Later discipline and holdover fields

- fe_discipline_state
- fe_lock_valid
- fe_holdover_ready

- fe_holdover_elapsed_s
- fe_holdover_est_error_ns
- fe_tuning_slope_ppb_per_v

11.4 Estimator diagnostics

- fe_qerr_used
- fe_estimator_confidence
- fe_sample_weight
- fe_reference_valid

12. Dashboard display plan

12.1 Existing top summary to preserve

- calibrated phase error
- 10-minute RMS jitter
- raw phase error
- phase bias
- auto-cal state
- Piksi-ZED RMS

12.2 New FE section

The dashboard should add a dedicated FE / Oscillator / Holdover section containing:

Cards

- FE state
- FE valid
- FE warmup time
- FE frequency error
- FE smoothed frequency error
- FE drift
- FE control voltage
- FE discipline state
- FE holdover readiness

Charts

- FE frequency error vs time
- FE drift trend vs time

- FE control voltage vs time
- later holdover estimate vs time if useful

Narrative analysis

- FE offline
- FE warming up
- FE stable in measure-only mode
- FE acquiring lock
- FE locked
- FE in holdover
- FE fault

13. Interface requirements

13.1 Hardware interfaces

- FE power input
- FE ground reference
- FE output measurement path to Teensy
- later FE control-voltage path
- optional control-voltage monitoring path

13.2 Software interfaces

- Pi-to-Teensy ZED timing-context data path
- Teensy telemetry packet extensions for FE metrics
- collector parsing and DB schema updates
- API exposure through existing endpoints such as /api/latest and /api/history

14. Phased implementation strategy

Phase 0 - Baseline preservation

Objective: Freeze and verify the current non-FE system baseline.

Hardware tasks

- leave present ZED/Pi/Teensy/Piksi wiring unchanged
- do not connect FE to active timing paths

Software tasks

- verify ZED tracking health

- verify chrony PPS lock
- verify Teensy telemetry stability
- verify dashboard consistency with DB
- refresh snapshot
- commit baseline state to git

Exit criteria

- known-good pre-FE state recorded

Phase 1 - FE bench bring-up

Objective: Power FE safely and verify stand-alone operation.

Hardware tasks

- identify FE pins for power, ground, and output
- power FE from stable source
- add local decoupling
- verify output presence
- observe warmup behavior

Software tasks

- none required in Pi/Teensy system

Exit criteria

- FE powers reliably and outputs stable signal

Phase 2 - Signal conditioning design

Objective: Define safe FE-to-Teensy measurement path.

Hardware tasks

- decide on comparator, buffer, divider, or direct digital conditioning
- verify Teensy-safe logic levels
- define wiring and grounding approach

Software tasks

- define Teensy measurement method
- define early telemetry fields

Exit criteria

- measurement path design finalized

Phase 3 - Teensy measure-only integration

Objective: Measure FE continuously without controlling it.

Hardware tasks

- connect FE measurement path to Teensy
- preserve existing ZED and Piksi paths unchanged

Software tasks

- add FE measurement support to Teensy firmware
- compute FE frequency error and validity
- add FE state logic for offline, warmup, and measure-only
- include FE values in telemetry packets

Exit criteria

- Teensy reports FE measurement data with no regression in existing timing pipeline

Phase 4 - Collector and database extension

Objective: Store FE metrics cleanly in the DB.

Hardware tasks

- none

Software tasks

- update live collector
- update repo collector
- extend DB schema
- preserve existing truth fields and semantics

Exit criteria

- FE values stored in DB and retrievable via API

Phase 5 - Dashboard FE monitoring

Objective: Expose FE state and measurements as a separate dashboard subsystem.

Hardware tasks

- none

Software tasks

- update live dashboard files
- update repo dashboard files
- add FE cards and charts
- sanitize invalid FE values
- keep current top timing summary unchanged

Exit criteria

- dashboard shows FE section clearly without source confusion

Phase 6 - Characterization and logging

Objective: Learn FE warmup, drift, and stability characteristics.

Hardware tasks

- maintain stable power and thermal conditions

Software tasks

- log FE over hours and days
- add smoothing and drift estimators
- evaluate warmup completion behavior
- optionally add holdover-readiness heuristic

Exit criteria

- FE warmup time and baseline stability behavior understood

Phase 7 - Control path instrumentation

Objective: Add safe FE control-voltage generation and monitoring.

Hardware tasks

- connect safe DAC or filtered PWM path
- protect control input
- add control-voltage sensing if possible

Software tasks

- add telemetry for commanded and measured control state
- add manual control mode support

Exit criteria

- FE control path can be driven manually and observed safely

Phase 8 - Manual tuning experiments

Objective: Characterize FE response to control input.

Hardware tasks

- apply small controlled EFC changes
- monitor for abnormal behavior

Software tasks

- log frequency response to steps
- estimate tuning slope

- estimate linear region and settling time

Exit criteria

- FE control sensitivity and safe range are known

Phase 9 - qErr-aware estimator implementation

Objective: Improve residual interpretation before closed-loop discipline.

Hardware tasks

- none

Software tasks

- extend Pi-to-Teensy ZED timing metadata path if needed
- ingest qErr or timing-uncertainty information
- weight or filter residuals based on uncertainty
- expose estimator confidence metrics

Exit criteria

- estimator uses qErr or equivalent timing uncertainty meaningfully and safely

Phase 10 - Closed-loop discipline prototype

Objective: Gently steer FE toward ZED truth.

Hardware tasks

- preserve all truth wiring
- use only the safe FE control path

Software tasks

- implement slow bounded control loop
- add acquire and locked states
- prevent aggressive second-to-second corrections
- expose discipline state and control values

Exit criteria

- stable lock achieved without oscillatory or noisy behavior

Phase 11 - Holdover modeling

Objective: Estimate expected FE behavior during loss of truth.

Hardware tasks

- none beyond stable operation

Software tasks

- simulate holdover behavior

- estimate time error growth
- add holdover readiness and estimated uncertainty

Exit criteria

- holdover error estimates are available and credible

Phase 12 - Operational holdover mode

Objective: Define deterministic behavior during real ZED timing loss.

Hardware tasks

- none unless later switching logic is added

Software tasks

- define holdover entry and exit rules
- freeze or transition control appropriately
- mark degraded state clearly in dashboard and reports

Exit criteria

- holdover mode behaves predictably and transparently

15. Verification strategy

15.1 Baseline verification

- verify ZED tracking state
- verify chrony PPS lock
- verify dashboard consistency

15.2 Bring-up verification

- verify FE output present
- verify startup repeatability
- verify thermal sanity

15.3 Measurement verification

- verify FE telemetry is continuous
- verify no regression in ZED/Piksi telemetry
- verify validity transitions behave as expected

15.4 Database and API verification

- verify FE fields populate correctly
- verify missing data is handled safely
- verify dashboard endpoints expose FE fields correctly

15.5 Control verification

- verify manual control does not exceed safe range
- verify observed tuning response is plausible
- verify control telemetry matches commands

15.6 Closed-loop verification

- verify corrections are bounded and slow
- verify lock is stable
- verify qErr-aware weighting does not destabilize behavior

15.7 Holdover verification

- verify holdover entry on loss of valid reference
- verify holdover indicators and error estimates are visible
- verify re-entry into tracking mode is deterministic

16. File and workflow implications

Live runtime files likely to be touched

- /home/pi/teensy_appliance/collector.py
- /home/pi/teensy_dash2/app.py
- /home/pi/teensy_dash2/static/app.js
- /home/pi/teensy_dash2/templates/index.html

Repo copies likely to be touched

- /home/pi/time-server/pi/teensy_appliance/collector.py
- /home/pi/time-server/pi/teensy_dash2/app.py
- /home/pi/time-server/pi/teensy_dash2/static/app.js
- /home/pi/time-server/pi/teensy_dash2/templates/index.html

Supporting files

- /home/pi/time-server/pi/send_zed_to_teensy.py
- /home/pi/time-server/monitoring/zed_monitor.py
- /home/pi/time-server/rebuild/dump_state.sh
- /home/pi/time-server/system_config/STATE_SNAPSHOT.txt

Workflow rules

- confirm live runtime path

- edit live file if runtime change is needed
- sync live copy back to repo copy
- refresh snapshot
- commit and push

17. Risks and design cautions

- R1 Do not let FE telemetry overwrite or redefine current corrected timing truth fields.
- R2 Do not make qErr the only control driver; it should be a weighting or correction aid, not the entire control strategy.
- R3 Do not use aggressive control updates that chase PPS noise.
- R4 Do not present FE as GNSS truth or as the primary second marker.
- R5 Do not allow FE subsystem faults to interfere with current ZED-based time operation.

18. Summary

This specification defines a safe and disciplined path for FE-5680A integration:

- preserve ZED as truth
- preserve Pi chrony on ZED PPS
- use Teensy to measure and later steer FE
- use FE first as a measured oscillator
- add dashboard visibility without source confusion
- incorporate qErr-aware estimation carefully
- add discipline and holdover only after characterization

Central design intent: ZED defines true time, Teensy measures and estimates timing behavior, FE provides local stability and later holdover, and qErr-aware estimation helps prevent over-correction.